

Telegram Mini App — Ludo & Chess with Points

Monorepo with `frontend/` (Telegram Mini App UI) and `backend/` (server + bot + game logic + points).

What's included

- Chess: online 1v1, moves validated with `chess.js`, real-time via Socket.io.
- Ludo: simplified but working online 1v1 (52-cell shared track, roll 6 to leave base, capture opponents).
- Points system: winner gets +10 points, stored server-side, bot commands `/points` and `/leaderboard` read the same data.
- Telegram auth: the backend verifies Telegram's `initData` (HMAC signature) so points can't be faked by calling the server directly.

1. Create your bot

1. Open Telegram, message @BotFather.
2. `/newbot` → follow steps → copy the bot token.

2. Run the backend locally

```
cd backend
npm install
```

```
cp .env.example .env
```

Edit `.env` :

```
BOT_TOKEN=your_token_here  
FRONTEND_URL=http://localhost:5173 # update  
after frontend is deployed  
PORT=3000
```

```
npm start
```

You should see `Server running on port 3000` and `Bot launched` .

3. Run the frontend locally

`frontend/` has no build step — it's plain HTML/CSS/JS.

Open `frontend/app.js` and set:

```
const BACKEND_URL = 'http://localhost:3000';
```

Serve it with any static server, e.g.:

```
cd frontend  
npx serve .
```

4. Deploy (so Telegram can load it — HTTPS required)

Backend → [Render.com](#) or [Railway.app](#)

1. Push this repo to GitHub.
2. New Web Service → connect repo → set root directory to `backend`.
3. Build command: `npm install` | Start command: `npm start`.
4. Add environment variables: `BOT_TOKEN`, `FRONTEND_URL` (fill this in after step below), `PORT` (usually auto-set).
5. Deploy → copy the generated URL, e.g. `https://your-app.onrender.com`.

Frontend → Vercel or Netlify

1. New Project → same repo → set root directory to `frontend`.
2. No build command needed (static site) — output directory: `.`
3. Before deploying, edit `frontend/app.js`:

```
const BACKEND_URL = 'https://your-app.onrender.com';
```

Commit and push.

4. Deploy → copy the URL, e.g. `https://your-app.vercel.app`.

Connect everything

1. Go back to backend host → update `FRONTEND_URL` env var to your Vercel URL → redeploy.
2. Message @BotFather → `/mybots` → your bot → Bot Settings → Menu Button → set the URL to your Vercel URL.
(Or use `/newapp` to register it as a proper Mini App.)

5. Test

Open your bot in Telegram → tap the menu button / “Play Games” → pick Chess or Ludo → open the same bot from a second Telegram account to get matched.

Notes & next steps

- Storage: `backend/db.js` uses a simple JSON file (`points.json`). Fine for testing; switch to MongoDB/PostgreSQL before real traffic (concurrent writes to a JSON file aren't safe at scale).
- Ludo rules: simplified (no exact-roll-to-finish, no home column stretch) so the multiplayer logic stays easy to follow. You can extend `backend/games/ludoGame.js` for full classic rules.
- Security: never trust points sent from the frontend — this backend always recalculates winners server-side and verifies Telegram's `initData` signature before touching the database.
- Scaling matchmaking/rooms: currently stored in memory (`rooms` object in `server.js`). If you deploy multiple backend instances, move this to Redis.